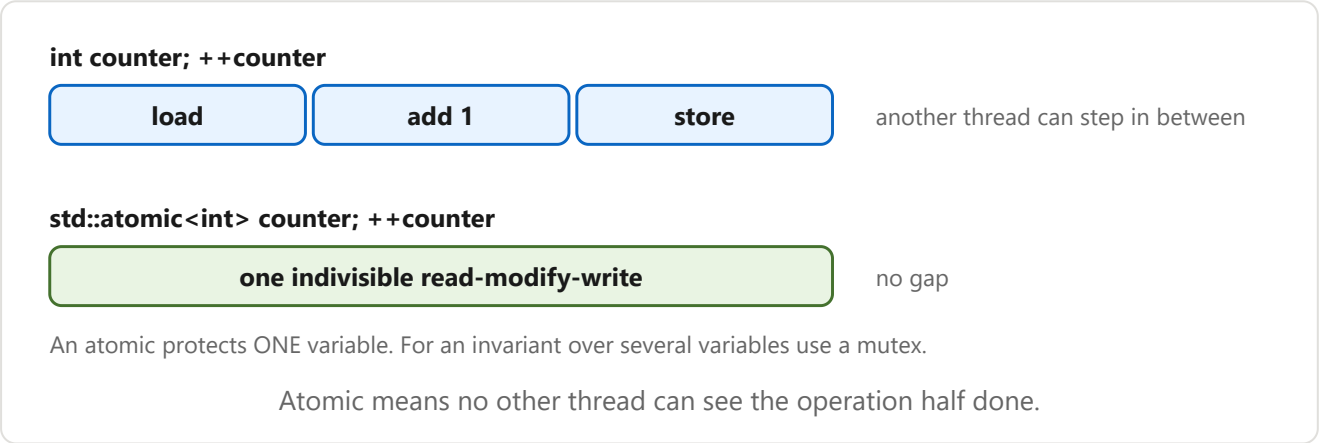
Atomics and Compare-and-Swap

Indivisible operations on one variable: `std::atomic`, compare-and-swap loops, `atomic_flag` and a spinlock, and the ABA problem. Assumes Thread Pools and Mutexes.

What an atomic is

`std::atomic<T>` makes operations on **one variable** indivisible, usually without blocking. The simulator shows the same two-thread increment, now with no gap to sneak into:



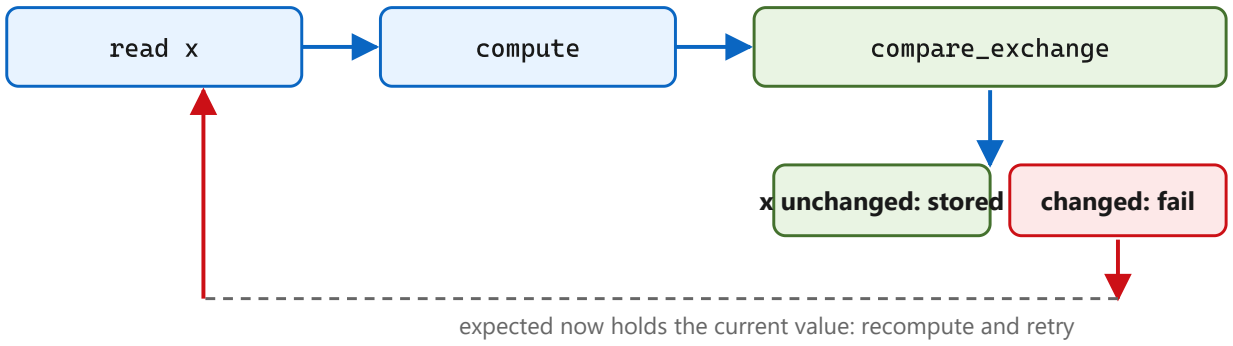

```
std::atomic<int> counter{0};
// one indivisible read-modify-write (same as fetch_add(1))
counter++;
int v = counter.load();          // atomic read
counter.store(5);                // atomic write
// write 9 and get the previous value in one step
int old = counter.exchange(9);
std::atomic<bool> stop{false};   // a very common use: a shutdown flag
```

USE AN ATOMIC FOR	DO **NOT** USE IT FOR
a counter, a flag, a swapped pointer, statistics	an invariant over **several** variables: use a mutex

> **VOLATILE IS NOT THREAD-SAFE** It only stops the compiler optimising an access away. No atomicity, no ordering. Use `std::atomic`.

Compare-and-swap

Compare-and-swap is the building block of every lock-free update. `compare_exchange(expected, desired)` means: **if the value still equals `expected`, set it to `desired`; otherwise load the current value into `expected` and fail.**



The CAS loop: read, compute, try to publish, retry if someone else won.

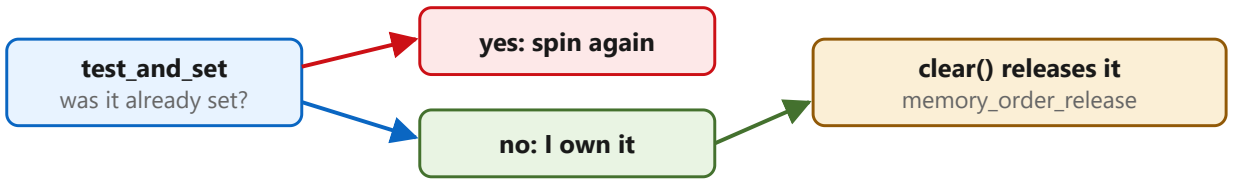
```
int expected = x.load();
while (!x.compare_exchange_weak(expected, expected * 2)) {
        // someone changed x: expected now holds the current value, so
    // recompute and retry
}
```

	`COMPARE_EXCHANGE_WEAK`	`COMPARE_EXCHANGE_STRONG`
spurious failure	**may** fail even when the value matched	fails only if the value really differed
use	inside a retry **loop** (cheaper on some CPUs)	a single attempt, no loop

Compiled: a CAS with a stale `expected` returned false and refreshed `expected` to the current value (99).

`atomic_flag` and a spinlock

`std::atomic_flag` is the one type guaranteed to be lock-free. With two calls it makes a **spinlock**: lock by looping on `test_and_set`, unlock with `clear`.



Four threads doing 50,000 increments each under this spinlock reached exactly 200,000 (compiled).

A spinlock trades CPU time for the cost of a context switch.

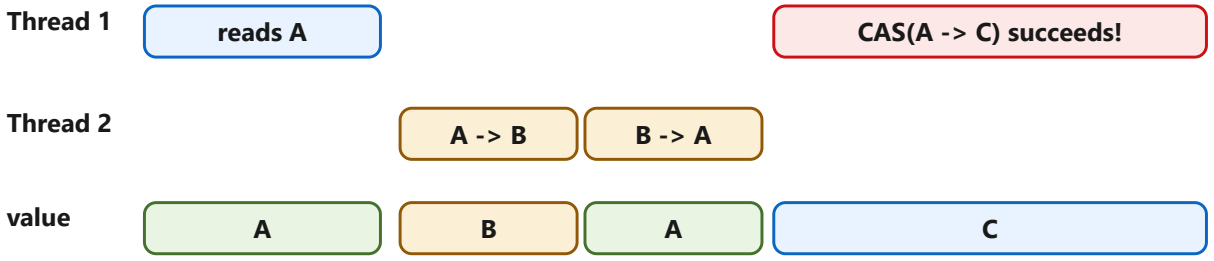
```
struct Spinlock {
    // starts clear in C++20
    std::atomic_flag f;
    void lock()   { while (f.test_and_set(std::memory_order_acquire)) { /*
spin */ } }
    void unlock() { f.clear(std::memory_order_release); }
};
```

SPINLOCK IS FINE	USE A MUTEX INSTEAD
very short critical sections on dedicated cores	the holder can be descheduled: spinners burn CPU for nothing
waiting is shorter than a context switch	long or blocking work under the lock

- `is_lock_free()` says whether a type uses real atomic instructions; big structs fall back to a hidden lock.

The ABA problem

A CAS only checks that the value is *equal* to what you read. If it went A to B and back to A in between, the CAS succeeds, but the structure around the value may have changed completely, for example a node that was freed and reused.



Example: a popped node is freed and reallocated at the same address, so the pointer looks unchanged.

A CAS compares values, not history.

FIX	IDEA
version counter	store a counter next to the value and compare both (a tagged pointer)
hazard pointers	threads announce what they are reading so nobody frees it
epoch-based reclamation	free only when no thread can still hold an old reference

The ABA problem and safe memory reclamation are why writing a lock-free *pop* is hard. Prefer a mutex, or a proven library.

Atomics in everyday code

Before the memory model, three everyday patterns cover most real uses of `std::atomic`.



None of them guards other data, so relaxed is enough for the counters.

A flag that publishes data needs release and acquire (next lesson).

Most uses of atomics are one of these three.

```
std::atomic<int> nextId{0};
// every caller gets a unique id
int newId() { return nextId.fetch_add(1, std::memory_order_relaxed); }

std::atomic<bool> stop{false};
// main calls stop.store(true) to ask it to finish
void worker() { while (!stop.load()) doWork(); }

std::atomic<long> bytes{0};
// statistics: only the final total matters
void onPacket(int n) { bytes += n; }
```

PATTERN	ORDER	WHY ENOUGH
unique ids, counters	`relaxed`	only atomicity matters; nothing else depends on the value
a stop flag that carries no data	default or `relaxed`	it only has to be seen eventually
a flag that says "the data is ready"	`release` store, `acquire` load	the data must be visible when the flag is (see the memory model lesson)

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
What an atomic is	Effective Modern C++	Item 40, Use std::atomic for concurrency, volatile for special memory	289 to 297
	C++ Concurrency in Action	5.1 Memory model basics (objects, memory locations, modification orders)	148 to 150
	C++ Concurrency in Action	5.2 Atomic operations and types in C++ (atomic_flag, atomic<bool>, pointers, integers, free functions)	151 to 164
	C++ Concurrency in Action	5.3 Synchronizing operations and enforcing ordering (synchronizes-with, happens-before, the memory orders, release sequences, fences)	165 to 196
	C++ Concurrency in Action	6.2 Lock-based concurrent data structures (a thread-safe stack and queue, fine-grained locks)	199 to 216
	C++ Concurrency in Action	7.1 to 7.3 Lock-free data structures: definitions, a lock-free stack, memory reclamation, the ABA problem	229 to 274
Compare-and-swap	Effective Modern C++	Item 40, Use std::atomic for concurrency, volatile for special memory	289 to 297